MATH DIRECTED READING PROGRAM / MEETING 6

Assemble a Transformer Block

Self-contained guided reading and problem sheet | Revision 3.0

Aim. Combine an attention operation, a position-wise network, residual connections, and layer normalization. Track which operations mix positions, which mix features, and which preserve the overall array shape.

Background and pacing. Matrix multiplication, coordinatewise ReLU, the chain rule, and attention as a weighted sum. The required formulas are restated. Discuss 6.1–6.2 before 6.3–6.5 if needed. The optional LayerNorm derivative is not required. This is a module with natural stopping points, not a timed assignment.

Using the self-checks. After each numbered problem, try its short self-checks with the worked solution covered. Definitions may stay open. Give a reason, not only a yes/no answer; these checks guide discussion rather than add a timed test.

6.1 A feed-forward network at every position

SETUP: ONE SMALL NETWORK, SHARED ACROSS ALL POSITIONS

Write $X \in \mathbb{R}^{n \times d_{\text{model}}}$ with one token representation per row. A **position-wise feed-forward network (FFN)** applies the same small network separately to each row. In this sheet it has two affine stages with ReLU in between:

$$\text{FFN}(X) = \text{ReLU}(XW_1 + \mathbf{1}_n b_1^\top)W_2 + \mathbf{1}_n b_2^\top,$$

where

$$W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, \quad b_1 \in \mathbb{R}^{d_{\text{ff}}}, \qquad W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}, \quad b_2 \in \mathbb{R}^{d_{\text{model}}}.$$

Here d_{ff} is the FFN's **hidden width**, $\mathbf{1}_n$ is an n-entry column of ones, and ReLU acts on every array entry by $\max(0, t)$. The product $\mathbf{1}_n b^\top$ repeats one bias row at every position. Software calls this repetition **broadcasting**. Such a network is also commonly called an **MLP** (**multilayer perceptron**). In this Transformer context, that term refers to the position-wise network, not the attention mechanism. The weights are shared across positions within this block; different Transformer blocks can have different weights.

For a column representation x_i, the equivalent single-position formula is

$$\text{FFN}(x_i) = W_2^\top \text{ReLU}(W_1^\top x_i + b_1) + b_2.$$

The transposes appear because the stored sequence uses rows. The mathematical map is the same one-hidden-layer construction as in Meeting 1.

Worked start. Let $d_{\text{model}} = 2$, $d_{\text{ff}} = 3$, and

$$W_1 = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \quad b_2 = 0.$$

For input row $(2, 3)$, the hidden pre-activation is $(2, -2, 2)$, and the hidden activation is $(2, 0, 2)$. The output row is $(4, 2)$.

6.1 / FEATURE COMPUTATION IS NOT POSITION MIXING

REVIEW: ROW INDICES AND FEATURE INDICES

In an $n \times d_{\text{model}}$ array, the first index names a position and the second names a feature coordinate. In XW_1, right multiplication changes feature coordinates within each row. In AV from attention, left multiplication by $A \in \mathbb{R}^{n \times n}$ combines information across position rows.

The phrase **position-wise** means that, when this operation is applied, its output at position i depends only on its input row i. That input row may already contain information gathered from other positions by an earlier attention operation.

Guided questions

(a) With the matrices in the worked start of 6.1, evaluate the FFN on the two-row array

$$X = \begin{pmatrix} 2 & 3 \\ -1 & 0 \end{pmatrix}.$$

Show explicitly where the same biases are added to both rows. Check the first row against the worked example.

(b) Write the ith output row for a general X. Explain why changing only input row $j \neq i$ cannot alter output row i of this FFN.

(c) Count the trainable scalar entries in W_1, b_1, W_2, b_2, obtaining $2d_{\text{model}}d_{\text{ff}} + d_{\text{ff}} + d_{\text{model}}$. Why is there no factor of n?

(d) If ReLU were replaced by the identity, expand the resulting expression into one affine map applied to every row. Relate this to the affine-composition calculation in Meeting 1.

(e) A permutation matrix P reorders rows and satisfies $P\mathbf{1}_n = \mathbf{1}_n$. Since reordering rows commutes with coordinatewise ReLU, prove $\text{FFN}(PX) = P\,\text{FFN}(X)$. What assumption about sharing parameters makes this possible?

A USEFUL ARCHITECTURE DISTINCTION

Attention combines information from different allowed positions using input-dependent coefficients. The FFN transforms the features at each position separately, using a shared nonlinear map. Both can contain trainable matrices, but they perform different kinds of computation. Neither phrase identifies a vocabulary probability distribution.

Translate back into machine-learning language. Explain the FFN once as a shared composition of affine maps and ReLU, and once as a position-wise MLP. Say exactly which coordinates can influence a single output row.

SELF-CHECK 6.1

1. A position-wise FFN receives an array whose first row already summarizes earlier tokens. Does being position-wise erase that context? Distinguish the current row dependency from the history of how the row was computed.
2. Keep feature widths fixed and increase the sequence length from 3 to 30. Do the FFN parameter count and the number of intermediate activations change in the same way?

6.2 Residual connections add an identity path

SETUP: ADD A LEARNED CHANGE TO THE INPUT

Given a differentiable function $F : \mathbb{R}^d \to \mathbb{R}^d$, define

$$R(x) = x + F(x).$$

This is a **residual connection**, also called a **skip connection**; F is the **residual branch**. The unchanged term x is the identity path. The two summands must have matching shapes. For sequence arrays, the same addition can be made in $\mathbb{R}^{n \times d_{\text{model}}}$ entry by entry.

The Jacobian J_F has entries $\partial F_i / \partial x_j$. For a scalar loss after R, if $g = \nabla_R \ell$ is its arriving column derivative, then $\nabla_x \ell = J_R(x)^\top g$. We calculate only at differentiable points; any ReLU coordinates involved are assumed away from zero.

Worked baseline. Setting F identically to zero makes R the identity map, not the zero map. This allows the architecture to represent "leave the input unchanged" through its skip path.

A useful small-change case. For the scalar branch $F(x) = ax$ with $a = 0.01$, its derivative is 0.01, while the residual map $R(x) = (1 + a)x$ has derivative 1.01. A small derivative along the learned branch need not remove the direct identity contribution. The next questions distinguish this mechanism from an unconditional guarantee.

Guided questions

(a) Differentiate $R_i(x) = x_i + F_i(x)$ to show

$$J_R(x) = I + J_F(x), \qquad \nabla_x \ell = g + J_F(x)^\top g.$$

Explain the two terms as contributions from two computational paths.

(b) If the residual branch is the FFN of 6.1, which setting of its output matrix and output bias makes the branch zero for every input? Must its first affine stage also be zero?

(c) Use $F(x) = -x$ to test the claim "an identity path guarantees a nonzero gradient." What is $R(x)$, and what is $J_R(x)$?

(d) Use $F(x) = x$ to test whether a residual connection always keeps gradient magnitude unchanged. What happens after composing this particular residual map L times?

(e) Explain why one cannot directly add $x \in \mathbb{R}^d$ to an arbitrary branch output in $\mathbb{R}^r$ when $r \neq d$. A projected skip path is another architecture choice, not the identity skip used here.

INTERPRETATION WITHOUT AN UNPROVED GUARANTEE

The residual parameterization can represent a perturbation of the identity, and its derivative has a direct identity contribution. These are precise algebraic observations. They do not prove that gradients never vanish or grow, or that every residual network trains successfully.

Translate back into machine-learning language. Translate "skip connection" into addition of two outputs, and "gradient flow along the skip path" into the identity term in the chain rule.

SELF-CHECK 6.2

1. For $F(x) = -0.05x$, compare the scalar derivatives of F and $x + F(x)$. What useful mechanism does this illustrate, and what does the example fail to guarantee for a general branch?
2. If a branch returns a three-coordinate vector but its skip input has two coordinates, is their direct sum defined? Which architecture choice would need to change?

6.3 Layer normalization, with its axis specified

SETUP: NORMALIZE FEATURES WITHIN ONE TOKEN

For one token representation $x \in \mathbb{R}^d$, where $d = d_{\text{model}}$, define

$$\mu(x) = \frac{1}{d}\sum_{j=1}^{d} x_j, \qquad v(x) = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu(x))^2.$$

These are the mean and variance of this vector's feature coordinates. The denominator is d, not $d - 1$. This is a deterministic calculation on the current vector, not an estimate requiring a dataset of independent observations. Fix a small positive number ε and set

$$u(x) = \frac{x - \mu(x)\mathbf{1}_d}{\sqrt{v(x) + \varepsilon}}, \qquad \mathrm{LN}(x) = \gamma \odot u(x) + \beta.$$

This is **layer normalization**, abbreviated **LayerNorm** or **LN**. The learned vectors $\gamma, \beta \in \mathbb{R}^d$ are featurewise scale and bias parameters. The mean and variance are recomputed from the input; ε is a fixed numerical stabilizer, not a learned weight. The symbol $\odot$ means coordinatewise multiplication.

For $X \in \mathbb{R}^{n\times d}$, apply this formula separately to each row, with the same γ, β shared across positions. The statistics use feature coordinates of one token, *not* all tokens and not all examples in a batch. This is the normalization axis chosen for this Transformer block.

Worked arithmetic. Take $x = (1, 3)^\top$ and, only to keep this example arithmetic simple, $\varepsilon = 3$. Then $\mu = 2$, $v = 1$, and $u = (-1/2, 1/2)^\top$. With $\gamma = (2, 1)^\top$ and $\beta = (1, -1)^\top$, the final LayerNorm output is $(0, -1/2)^\top$. Its coordinate mean is no longer zero. The large example value of ε is not a recommended training setting.

WHAT "NORMALIZED" DOES AND DOES NOT DESCRIBE

Mean subtraction gives u zero coordinate mean. Its coordinate variance is not exactly one when $\varepsilon > 0$. Learned scale and bias can change both the mean and variance again. LayerNorm is not softmax: its outputs need not be nonnegative or sum to one.

Before the questions. Identify four kinds of objects in the formula: the input x, computed statistics μ, v, trainable parameters γ, β, and fixed setting ε. They have different roles even though all appear in one expression.

6.3 / VERIFY THE NORMALIZATION, NOT A SLOGAN

Guided questions

(a) Compute the coordinate mean and variance of $u(x)$ to prove

$$\frac{1}{d}\sum_j u_j = 0, \qquad \frac{1}{d}\sum_j u_j^2 = \frac{v(x)}{v(x)+\varepsilon}.$$

For constant $x = c\mathbf{1}_d$, compute u and $\mathrm{LN}(x)$. Why is the formula still defined?

(b) For fixed $\gamma, \beta, \varepsilon$, prove $\mathrm{LN}(x + c\mathbf{1}_d) = \mathrm{LN}(x)$ for any scalar c. Keep track of both the mean and variance.

(c) Let $a > 0$. Compute $u(ax + c\mathbf{1}_d)$ and compare it with $u(x)$. Is positive-scale invariance exact for fixed $\varepsilon > 0$? If one formally sets $\varepsilon = 0$ and assumes $v(x) > 0$, what changes?

(d) Count trainable parameters in one LayerNorm with learned γ, β. Are $\mu(x)$ and $v(x)$ trainable scalar entries? Explain why sharing γ, β does not mean sharing one mean and variance across all tokens.

(e) If one were to compute statistics across all token positions, could changing a future token change the normalized representation of an earlier token? Explain why per-token feature normalization avoids that particular way of leaking future information.

Translate back into machine-learning language. Describe LayerNorm as a specified coordinatewise mean/variance calculation followed by learned scaling and shifting. Avoid replacing that formula with the false claim that its final output always has mean zero and variance one.

SELF-CHECK 6.3

1. With fixed $\gamma, \beta, \varepsilon$, compare LayerNorm of $(1, 3)^\top$ and $(11, 13)^\top$. Which statistics change, and is the output changed?
2. For $X \in \mathbb{R}^{n\times d}$, you accidentally compute one mean over *all nd* entries. Does this still implement the per-token LayerNorm defined here? Could a changed future row affect an earlier normalized row?

6.4 Put the components in a pre-LN block

SETUP: FIX ONE ARCHITECTURE BEFORE COMPOSING IT

A **Transformer block** combines attention and a position-wise FFN, with normalization and residual connections. Here choose a **pre-layer-normalization (pre-LN)** block: normalize the input to each residual branch *before* evaluating that branch. With two separately parameterized LayerNorms,

$$U = X + \mathrm{MHA}(\mathrm{LN}_1(X)), \qquad Y = U + \mathrm{FFN}(\mathrm{LN}_2(U)).$$

All four arrays $X, \mathrm{LN}_1(X), U, Y$ have shape $n \times d_{\text{model}}$, as does $\mathrm{LN}_2(U)$. A **sublayer** here means the attention or FFN part inside the block. "Pre" describes normalization's position relative to its sublayer and skip addition.

Attention definition for this block. For an input $T \in \mathbb{R}^{n\times d_{\text{model}}}$, assume n_{heads} divides d_{model}. Use that many bias-free heads with $d_k = d_v = d_{\text{model}}/n_{\text{heads}}$ and

$$\begin{aligned} Q_a &= TW_Q^{(a)}, \quad K_a = TW_K^{(a)}, \quad V_a = TW_V^{(a)}, \\ O_a &= \mathrm{softmax}_{\text{row}}(Q_a K_a^\top/\sqrt{d_k} + M)V_a, \\ \mathrm{MHA}(T) &= [\,O_1 \mid \cdots \mid O_{n_{\text{heads}}}\,]W_O. \end{aligned}$$

Here $W_Q^{(a)}, W_K^{(a)}, W_V^{(a)} \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. Concatenation joins feature columns. Row-wise softmax normalizes over allowed key positions. For this block choose the inclusive causal mask $M_{ij} = 0$ for $j \le i$ and $M_{ij} = -\infty$ for $j > i$.

The FFN is the ReLU network of 6.1 with both biases. Each LayerNorm has its own learned scale and bias vectors. We omit **dropout**, a training operation that randomly zeros some activations. No positional scheme or vocabulary head is included in this block's parameter count.

ANOTHER STANDARD CHOICE: POST-LN

A **post-LN** block instead has

$$U = \text{LN}_1(X + \text{MHA}(X)), \qquad Y = \text{LN}_2(U + \text{FFN}(U)).$$

These are different functions, not two names for the same expression. The original Transformer uses the post-LN arrangement; pre-LN is another established choice. A stack of pre-LN blocks is commonly followed by a final LayerNorm before prediction; that final layer is outside the individual block defined here.

Translate back into machine-learning language. Read the pre-LN formula aloud in computation order. Explain which value is supplied to the branch and which value travels unchanged along the skip path.

6.4 / VERIFY THE COMPOSITION

Guided questions

(a) Give the shape of every intermediate array, including the concatenated heads and the FFN hidden activations. Why are both residual additions well-defined?

(b) Count the block's trainable parameters by adding bias-free attention, the biased FFN, and two LayerNorms. Derive

$$4d_{\text{model}}^2 + 2d_{\text{model}}d_{\text{ff}} + d_{\text{ff}} + 5d_{\text{model}}.$$

Evaluate it for $d_{\text{model}} = 4$, $d_{\text{ff}} = 8$, and $n_{\text{heads}} = 2$. Do not count intermediate activations or an external vocabulary head.

(c) If both residual branches are identically zero, what does the pre-LN block compute? Under the same zero-branch condition, must the post-LN block compute the identity? Use a constant input row to test the claim.

(d) Suppose two input arrays agree through row i and may differ after i. Explain, in order, why their first i output rows agree: per-token normalization, causal attention, first residual addition, position-wise FFN, second residual addition. Assume all input representations up to i are indeed the same.

WARM-UP: CHECK THE COMPUTATION ORDER

Let $n = d_{\text{model}} = 2$, use one head, and take

$$X = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \quad W_Q = W_K = 0, \quad W_V = W_O = I_2.$$

In LN_1, choose $\gamma = (1, 1)^\top$, $\beta = 0$, and $\varepsilon = 3$ solely for simple arithmetic. Set the FFN's $W_2 = 0$ and $b_2 = 0$, so its entire output vanishes.

Compute $\text{LN}_1(X)$, the zero score matrix, the causal attention matrix, and then U, Y. The first row may use only itself; the second may average both normalized rows. Is it necessary to calculate $\text{LN}_2(U)$ numerically to know the final Y in this specially chosen example? This checks the wiring but deliberately disables content-dependent scores and the FFN; Problem 6.5 activates them.

A BLOCK IS NOT YET A COMPLETE LANGUAGE MODEL

Y contains feature representations, not vocabulary probabilities. For a prediction, write a final position representation as a column $h_i = Y_{i,:}^\top \in \mathbb{R}^{d_{\text{model}}}$, form logits $z_i = W_{\text{out}} h_i + b$, and apply softmax over token classes. Training can then use the observed next-token cross-entropy from Meeting 3. A complete model also specifies token embeddings, position handling, the stack of blocks, and any final normalization. Those choices are not silently part of this one block.

Translate back into machine-learning language. Explain attention, FFN, residual addition, and LayerNorm as four different operations on arrays. Then identify the separate operation that would turn a final representation into next-token probabilities.

SELF-CHECK 6.4

1. Both residual branches are zero. Compare pre-LN and post-LN on a constant input row with normalization bias zero. Which block preserves the row and why?
2. Does knowing the block output $Y \in \mathbb{R}^{n \times d_{\text{model}}}$ already supply a vocabulary distribution? Identify the extra map and its output dimension.

6.5 From an active block to a prediction and loss

SETUP: EVERY COMPONENT NOW PARTICIPATES

Use $n = d_{\text{model}} = 2$, one head, $d_k = d_v = 2$, and

$$X = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \qquad W_Q = W_K = W_V = W_O = I_2.$$

Use the inclusive causal mask. In *both* LayerNorms, set $\gamma = (1, 1)^\top$, $\beta = 0$, and $\varepsilon = 3$ for arithmetic only. These two layers have separate parameters even though their values coincide here.
The FFN has $d_{\text{ff}} = 2$, $W_1 = W_2 = I_2$, and $b_1 = b_2 = 0$, so $\text{FFN}(T) = \text{ReLU}(T)$. The block is

$$T = \text{LN}_1(X), \quad O = \text{MHA}(T), \quad U = X + O, \quad Z = \text{LN}_2(U), \quad Y = U + \text{ReLU}(Z).$$

Here Z names the second normalized array, *not* class logits; we use lower-case z for logits below. All arrays in this line have shape 2×2.

Worked first step. As in the warm-up, $T = \frac{1}{2}\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$. The projections are now identities rather than zero matrices, so $Q = K = V = T$ and the score matrix is $S = TT^\top/\sqrt{2}$.

Guided questions: complete the block

(a) Compute S. Mask and normalize each row to obtain A, then compute $O = AT$ and $U = X + O$. To simplify the second row, set

$$a = \frac{1}{1 + e^{1/\sqrt{2}}}, \qquad r = \frac{3}{2} - a.$$

Show that the rows of U are $(3/2, -3/2)$ and $(-r, r)$. Which first-row attention weight is forced by the mask rather than by the scores?

(b) For $u = (b, -b)^\top$, verify $\mathrm{LN}(u) = (b, -b)^\top / \sqrt{b^2 + 3}$ under this example's settings. Apply this to each row of U to find Z. Which coordinates survive the FFN's ReLU? Compute Y; neither FFN output row is zero.

Optional arithmetic aid. $a \approx 0.33024$ and $r \approx 1.16976$. Keep exact expressions until the last step or use a calculator. The point is to track operations, not to do long decimal arithmetic by hand.

6.5 / REPRESENTATIONS BECOME CLASS SCORES ONLY AT THE HEAD

SETUP: A THREE-CLASS HEAD AND ONE OBSERVED LABEL

Use the final column representation $h = Y_{2,:}^\top$ and attach the output head

$$z = W_{\text{out}} h + b, \qquad W_{\text{out}} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad b = 0, \qquad p_j = \frac{e^{z_j}}{\sum_{k=1}^{3} e^{z_k}}.$$

There is no extra final LayerNorm in this toy classifier. The observed class is the integer $y = 2$, with one-hot vector $\mathbf{y} = (0, 1, 0)^\top$, and the loss is $\ell = -\log p_2$. This is a prediction head attached to one block, not a specification of an entire language model.

For this loss, the previously derived rule is $\nabla_z \ell = p - \mathbf{y}$. Through the affine head, the derivative with respect to its input is $W_{\text{out}}^\top (p - \mathbf{y})$.

Guided questions: connect prediction and training

(c) Use your Y to compute h, z, p, ℓ. Which class has the greatest predicted probability? Keep track of the difference between a two-feature representation and a three-class prediction.

(d) Find the shapes and values of $\nabla_{W_{\text{out}}} \ell$ and $\nabla_b \ell$ in terms of p and h. If the whole block is trainable, which two-coordinate derivative must be propagated back into its second output row?

(e) Now deliberately **freeze** the block: hold all its parameters fixed while fitting the head. With h fixed, use the two head gradients to take one simultaneous step with learning rate $\eta = 0.1$. Recompute z^+, p^+, and the new loss. You may use

$$z^+ = z - \eta(\|h\|_2^2 + 1)(p - \mathbf{y}),$$

which follows by substituting the updated head matrix and bias. Which trainable component has changed, and which attention weights have stayed fixed?

Translate back into machine-learning language. You have carried out a nonzero attention computation, both normalizations, a nonzero FFN, two residual additions, a classification prediction, and a head update. Training the attention itself would also propagate derivatives through the block, as illustrated in the simpler one-parameter example of 4.5.

SELF-CHECK 6.5

1. Change only the target from class 2 to class 1 without updating any parameter. Which of A, U, Y, z, p changes? Which loss derivative changes? Explain the difference between a forward computation and its supervision.
2. In part (e) only the head was updated. Did the attention pattern learn from that step? Where would derivatives have to continue in order to train the block?

Optional extension 6.E: the LayerNorm Jacobian

6.E / THE JACOBIAN OF THE NORMALIZED VECTOR

Here is a compact calculus exercise, not required for constructing the block. Write

$$P = I - \frac{1}{d}\mathbf{1}_d\mathbf{1}_d^\top, \qquad c = Px, \qquad s = \sqrt{\|c\|_2^2/d + \varepsilon}, \qquad u = c/s.$$

P is the ordinary linear projection onto the zero-sum coordinate subspace. By differentiating c and s in a direction w, show

$$J_u = \frac{P}{s} - \frac{cc^\top}{d\,s^3}, \qquad J_{\mathrm{LN}} = \mathrm{diag}(\gamma)J_u.$$

Check $J_u\mathbf{1}_d = 0$. If $c \neq 0$, show $J_u c = (\varepsilon/s^3)c$. These respectively reflect exact shift invariance and the failure of exact scale invariance when $\varepsilon > 0$.

SELF-CHECK 6.E

1. Which direction is exactly annihilated by J_u for every $\varepsilon > 0$? If the centered vector $c \neq 0$, is the direction c also annihilated? Relate your answer to shift versus scale invariance.

Discussion checkpoint and selected hints

READY TO MOVE ON WHEN...

Complete the nonzero block calculation in 6.5 and reach a class prediction, a loss, and a head update. Identify which operations mix positions, which are position-wise, and which arrays are parameters. Explain why the block itself is not a vocabulary distribution and why freezing it prevents its attention parameters from learning during the head-only step.
You may consult definitions and notation. Explain the choices without following a worked solution line by line. If a self-check exposes a gap, return to that problem's setup; optional extensions do not block progress.

CHECK YOUR TRANSLATIONS

Position-wise FFN: a shared nonlinear map applied to each token's features. Residual connection: add a shape-compatible branch to its input. LayerNorm: normalize features of each token, then apply learned scale and bias. Pre-LN: normalize inside each residual branch before its sublayer.
Attention's W_O returns concatenated head features to model width. A vocabulary head's W_{out} maps final features to logits. LayerNorm and softmax normalize different things and produce different kinds of outputs.

Suggested discussion. Reconstruct the two-line pre-LN block formula and annotate every array's shape. Give one example disproving an overstatement about residual gradients or LayerNorm output statistics.

6.1(c). Count the four arrays separately: $d_{\mathrm{model}}d_{\mathrm{ff}}$, d_{ff}, $d_{\mathrm{ff}}d_{\mathrm{model}}$, and d_{model}.

6.2(c)–(d). The Jacobian can be 0 for $F = -\mathrm{id}$ or $2I$ for $F = \mathrm{id}$; the skip path alone is not a norm bound.

6.3(c). For $a > 0$, the centered numerator scales by a, while the denominator is $\sqrt{a^2v + \varepsilon}$, not $a\sqrt{v + \varepsilon}$.

Optional Jacobian. Use $P^\top = P$, $Pc = c$, and $Ds[w] = c^\top w/(ds)$.

6.4(b). Two LayerNorms contribute $4d_{\mathrm{model}}$ parameters in total. Under the stated bias conventions, the example count is 156.

Small forward pass. The normalized rows are $(1/2, -1/2)$ and $(-1/2, 1/2)$. Causal attention gives rows $(1, 0)$ and $(1/2, 1/2)$ of attention weights.

Optional reading: reconnect with standard sources

[1] Vaswani et al., *Attention Is All You Need* (2017). §3.1: original block and post-LN arrangement; **§3.2.2**: multi-head attention; **§3.3**: position-wise feed-forward network.

[2] Ba, Kiros, and Hinton, *Layer Normalization* (2016). §3, the normalization definition. For the exact epsilon and featurewise affine convention used here, compare the formula and `normalized_shape` description in the PyTorch `LayerNorm` documentation.

[3] Xiong et al., *On Layer Normalization in the Transformer Architecture* (ICML 2020). Paper and architecture discussion: compare pre-LN with post-LN. Its optimization analysis is not required.

[4] Prince, *Understanding Deep Learning* (2023). §11.2, residual connections and residual blocks; **§12.4**, Transformers. D2L §11.7.2–§11.7.3 gives a second exposition of position-wise FFNs and residual/normalization components.

Readings supply context, not missing definitions. Exercises and numerical examples are original to this DRP. Prince locators refer to the 2023 first-edition organization; D2L locators refer to its 1.0.3 online edition. Use section titles when editions differ.